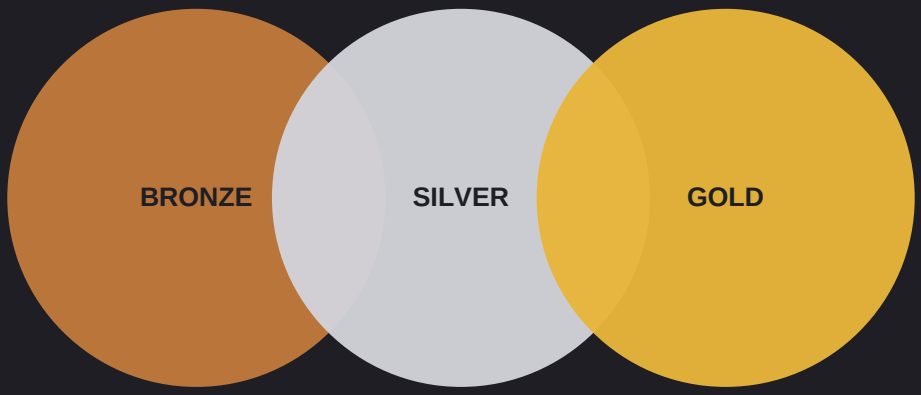

crudo · limpio · listo para consumir

PRÁCTICA 1 · UNIVERSIDAD DE LEÓN

Data Lake con Delta y limpieza de datos

Las decisiones que tomé en cada ejercicio: qué hice, por qué lo hice así y cómo interpreto los resultados.

Práctica
Práctica 1 · Data Lake con Delta

Fecha
octubre de 2026

Presentado por
Sofía Fuentes Crespo

Titulación
GIDIA · Universidad de León

Índice

Puntos importantes

Resumen de lo que hice en cada clase y de lo que me llamó la atención. El código sigue el patrón MVC: `modelo/` tiene las funciones (descargar, limpiar, Delta, MERGE, cruzar, exportar), `vista/` lo que se imprime y `controlador/` un archivo por clase. Se ejecuta con `python main.py clase1`, `clase2` o `clase3` (y un ejercicio suelto, por ejemplo `clase3 ej10`). Uso PySpark 4.2.0 y delta-spark 4.4.1.

01
CLASE 1 · BRONZE
página 3

02
CLASE 2 · DELTA LAKE
página 4

03
CLASE 3 · SILVER Y GOLD
página 5

04
PROBLEMAS Y DUDAS
página 6

Clase 1: capa bronze

36	18.396	14.652
empresas del IBEX-35 descargadas	cotizaciones en bronze/ibex	filas de sentimiento en bronze

Ej1 Descarga y bronze de precios

La función `descargar_historico` baja el histórico de un ticker con yfinance (sin ajustar precios) y define el esquema a mano para controlar los tipos. Aplicada a las 36 empresas da 18.396 filas (511 sesiones cada una), que guardo en Parquet con `coalesce(1)`, porque con tan pocos datos las 16 particiones por defecto solo añaden tareas diminutas. Date y Ticker no admiten nulos; Open, High, Low, Close y Volume sí, porque Yahoo a veces no informa; Dividends y Stock_Splits van con 0.0. Al leer solo Ticker, Date y Close, Spark abre esas tres columnas y se salta el resto (column pruning), algo que un CSV no permite.

Ej2 Sentimiento y el problema del append

El CSV llega con todo como texto (no guarda tipos) y con separador `;` (con coma salía una sola columna). Lo guardé en `bronze/sentimiento` con 14.652 filas. Al repetir la carga en modo `append` pasó a 29.304: Parquet no comprueba duplicados, así que repetir la carga ensucia la tabla. Lo dejé limpio con `overwrite`. El MERGE de la clase siguiente resuelve justo esto.

Clase 2: Delta Lake

+468
filas nuevas en el primer MERGE

0
filas nuevas en el segundo MERGE

5
versiones de la tabla tras el RESTORE

Ej3 y Ej4 Convertir a Delta y MERGE

Con `convertToDelta` las carpetas Parquet pasan a Delta sin reescribir datos: solo se crea `_delta_log` con la versión 0. Delta no admite espacios en columnas, así que renombré `Stock Splits` a `Stock_Splits`. Después descargué hasta el 20 de octubre y lo incorporé con `MERGE` sobre (Ticker, Date):

Ejecución	Filas antes	Filas después	Nuevas
MERGE 1	18.396	18.864	468
MERGE 2	18.864	18.864	0

La segunda ejecución no cambia nada: es idempotente, al contrario que el append del Ej2c.

Ej5 y Ej6 Esquema, DELETE, historial y RESTORE

Al añadir `Close_mas5` y hacer append, Delta lo rechaza (`DELTA_METADATA_MISMATCH`): es el schema enforcement. Con `mergeSchema` la columna se incorpora y las filas viejas quedan a nulo. Como reescribí la tabla entera, también dupliqué filas (37.728). Luego borré BBVA con `DELETE` (36.680 filas) y recuperé el estado del Ej4 con `restoreToVersion(1)`: 18.864 filas y el esquema original. `versionAsOf` solo lee una versión antigua; RESTORE además deja la tabla así y crea una versión nueva, la 4.

0. **CONVERT**
   Conversión de Parquet a Delta
1. **MERGE**
   Días nuevos (18.864 filas)
2. **WRITE**
   Append con mergeSchema (37.728 filas)
3. **DELETE**
   Borrado de BBVA (36.680 filas)

Clase 3: silver y gold

18.864
cotizaciones en la tabla de hechos

12
sectores tras limpiar 48 nombres

504
cotizaciones sin sentimiento

Ej7 Datos del experto

El histórico no trae el sector y el sentimiento va por sector, así que uso `Ticker.info` de Yahoo para sacar nombre, sector, industria, país, divisa, capitalización, beta, PER y rentabilidad por dividendo de cada empresa. Hay que mirarlo con cuidado: Cellnex sale como Real Estate, el PER viene vacío en algunas y `dividendYield` llega ya en porcentaje.

Ej8 y Ej9 Silver

Mi CSV no es como el del enunciado (ancho, comas decimales, vacíos, sectores en español): es largo, sin esos defectos y en inglés. Su defecto real era que los sectores aparecían escritos de 48 formas distintas para solo 12. La función de limpieza normaliza el sector, tipa con `try_cast`, quita nulos y duplicados y deja una fila por (Fecha, Sector); además cubre los casos del enunciado por si los datos cambian. Los precios ya venían limpios, así que solo se validan. Los datos del experto van a una dimensión aparte, `dim_empresa` (36 filas).

Ej10 a Ej13 Gold, anti-joins, indicadores y exportación

La estrella tiene tres tablas Delta:

Tabla gold	Filas	Contenido
fact_cotizacion	18.864	Claves, precios, volumen y sentimiento
dim_empresa	36	Nombre, sector, industria, país
dim_fecha	750	Calendario completo

El cruce es un `left join` por (Date, Sector), así no se pierde ninguna cotización. Con `left_anti` vi que 504 precios (1-20 de octubre) se quedan sin sentimiento porque el CSV acaba el 30 de septiembre, y que hay 9.042 filas de sentimiento sin precio: 1.277 son de Crypto, sin empresas en el IBEX-35, y el resto son fechas anteriores a octubre de 2024. No relleno huecos para no inventar datos. Añadí tres indicadores con funciones de ventana (variación diaria, media móvil de 20 sesiones y rango diario) y exporté la estrella a Parquet sin `_delta_log`, que Power BI no entiende: 733 KB el hecho y 6 KB cada dimensión.

Problemas y dudas

Lo que más tiempo me llevó: el separador del CSV, el espacio en `Stock Splits` y que Spark se colgara al arrancar en Windows, que arreglé fijando `spark.driver.host` y `spark.driver.bindAddress` a `127.0.0.1`.

Duda para el profesor: el enunciado describe un CSV ancho con comas decimales, vacíos y sectores en español, y el mío no tiene nada de eso. Quiero confirmar que es el mismo que usará para corregir.

Uso de IA: he usado Claude (Anthropic) como apoyo para escribir y depurar el código y para redactar esta memoria, y lo indico en los comentarios del código. Las pruebas las ejecuté yo y las cifras salen de esas ejecuciones.